



**Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

КАФЕДРА ИУ5

**Дисциплина:
Технологии машинного обучения
Отчет по лабораторной работе**

**Выполнил:
Емельянов А.К.
ИУ5-63Б
(Индекс группы)**

**Проверил: Гапанюк Ю.Е.
(Фамилия И.О.
преподавателя)**

Москва – 2026



Лабораторная 5: Ансамблевые модели (Bagging / AdaBoost / Gradient Boosting)

Датасет: smart_healthcare_dataset.csv

Задача: классификация

Целевая переменная: heart_disease (0/1)

Метрика для сравнения моделей: F1-score

```
In [1]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier, Ada
from sklearn.metrics import f1_score

pd.set_option("display.max_columns", 200)
```

```
In [2]: DATA_PATH = "smart_healthcare_dataset.csv"

df = pd.read_csv(DATA_PATH)
df.head()
```

```
Out[2]:
```

	age	gender	bmi	exercise_level	smoking	alcohol	blood_pressure	cholesterol
0	56	Male	22.6	2	1	1	169	160
1	69	Female	28.2	0	1	1	136	135
2	46	Female	25.1	1	0	1	142	145
3	32	Female	18.0	0	0	1	173	170
4	60	Female	20.1	2	1	0	130	130

```
In [3]: # Пропуски (если есть)
missing = df.isna().sum().sort_values(ascending=False)
missing[missing > 0].head(20)
```

```
Out[3]: Series([], dtype: int64)
```

```
In [4]: target = "heart_disease"

X = df.drop(columns=[target])
```

```

y = df[target]

print("Размер X:", X.shape)
print("Доли классов:")
print(y.value_counts(normalize=True).rename("share").to_frame().join(y.value_c

categorical_features = X.select_dtypes(include=["object"]).columns.tolist()
numeric_features = [c for c in X.columns if c not in categorical_features]

categorical_features, len(numeric_features)

```

Размер X: (5000, 15)

Доли классов:

	share	count
heart_disease		
0	0.7002	3501
1	0.2998	1499

Out[4]: (['gender'], 14)

```

In [5]: X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=42,
        stratify=y,
    )

X_train.shape, X_test.shape

```

Out[5]: ((4000, 15), (1000, 15))

```

In [6]: # Предобработка: заполнение пропусков + ONE для категорий + стандартизация чис
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ],
    remainder="drop",
)

```

```

In [7]: # Ансамблевые модели
        # 2 модели группы бэггинга: RandomForest + ExtraTrees
        # AdaBoost
        # Gradient Boosting

```

```

models = {
    "RandomForest": RandomForestClassifier(
        n_estimators=300,
        random_state=42,
        class_weight="balanced",
        n_jobs=-1,
    ),
    "ExtraTrees": ExtraTreesClassifier(
        n_estimators=500,
        random_state=42,
        class_weight="balanced",
        n_jobs=-1,
    ),
    "AdaBoost": AdaBoostClassifier(
        n_estimators=300,
        learning_rate=0.5,
        random_state=42,
    ),
    "GradientBoosting": GradientBoostingClassifier(
        random_state=42,
    ),
}

pipelines = {
    name: Pipeline(steps=[("preprocess", preprocess), ("model", model)])
    for name, model in models.items()
}

list(pipelines.keys())

```

Out[7]: ['RandomForest', 'ExtraTrees', 'AdaBoost', 'GradientBoosting']

```

In [8]: def fit_predict_f1(pipe, X_train, y_train, X_test, y_test):
        pipe.fit(X_train, y_train)
        y_pred = pipe.predict(X_test)
        return f1_score(y_test, y_pred)

scores = {}
for name, pipe in pipelines.items():
    scores[name] = fit_predict_f1(pipe, X_train, y_train, X_test, y_test)

pd.Series(scores, name="F1").sort_values(ascending=False).to_frame()

```

Out[8]:

	F1
AdaBoost	0.988314
GradientBoosting	0.976744
RandomForest	0.947899
ExtraTrees	0.945946

Вывод

- Выполнено разбиение на train/test, предобработка (пропуски + One-Hot для gender).
- Обучены ансамбли: **RandomForest**, **ExtraTrees** (группа бэггинга), **AdaBoost**, **GradientBoosting**.
- Качество сравнено по **F1-score** (таблица выше).